

Apellido y Nombres	Legajo	# de Hojas	Profesor

Normas Generales

Lea detenidamente cada pregunta y consulte las dudas de que pudieran surgir. La correcta interpretación del enunciado forma parte de la evaluación.

En caso de realizar el examen en papel, numere las hojas entregadas. Complete en la primera hoja la cantidad total de hojas entregadas. Debe identificarse cada hoja con: Nombre, Apellido, Legajo.

Por favor entregar esta hoja junto al resto de las hojas realizadas.

Enunciado (Tiempo 90 minutos)

Se requiere desarrollar una aplicación “servidor” basada en una arquitectura cliente-servidor utilizando sockets.

- El programa cliente no hay que desarrollarlo, se incluye su código fuente en el anexo “Código Fuente del Cliente”
- El servidor deberá incluir lo siguiente:

1. La dirección IP será la local (localhost o 127.0.0.1)
2. El puerto será el 8000 por default. Además también podrá ser pasado opcionalmente como argumento del main(), como se indica a continuación:

`$. /server port [8001]`

Si no se pasa ningún argumento, se utiliza el default, caso contrario se utiliza el pasado desde la línea de comandos. En ese caso verificar que se hayan recibido tal cual se indica en el ejemplo: la palabra “port” y luego el numero de puerto 8001 (ejemplo)

3. El servidor deberá ser “multi-thread”, es decir por cada conexión aceptada, lanzar un thread.
4. La cantidad máxima de conexiones simultaneas no deberá ser más de 10.
5. El “protocolo” entre el cliente y el servidor es el siguiente:
 - El cliente envía la siguiente información:
 - Un integer con la longitud del mensaje a enviar. Tener en cuenta que se enviará con “network byte order”.
 - Luego el string con la siguiente información:

Apellido y Nombres	Legajo	# de Hojas	Profesor

- "<hostname>;<porcentaje de cpu ocupada>;<porcentaje de memoria ocupada>"

Ejemplo:
host0;40;64

- El servidor al recibir esta información la pondrá en un array de estructuras como se indica a continuación:

```
#define HOSTNAME_LEN 64
#define STATUS_SIZE 10

typedef struct
{
    char hostname[HOSTNAME_LEN];
    int cpu;
    int mem;
} status_t;

status_t client_status[STATUS_SIZE];

int main(int argc, char **argv)
{
    return 0;
}
```

- Además, se deberá crear un thread particular, que solo se dedique a revisar que el array no este lleno, cuando detecte que el array este lleno, deberá:
 - Generar un archivo "log" (texto) solo con los elementos cuya cpu supere el 50% y/o la memoria supere el 50%. Esta información se tiene que obtener del array.
 - Luego vaciar el array

El nombre del archivo log será "status.log y el formato será el siguiente:

<Fecha>;<hora>;<hostname>; cpu=%<value>;memory=%<value>

Si el array esta lleno, se tienen que rechazar las nuevas conexiones.